

PROJECT REPORT: FACE AND HAND DETECTION SYSTEM

BY – SAURABH RANJAN

COURSE: FUNDAMENTALS IN ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING (CSA 2001)

1. Project Objective

The objective of this project is to develop a real-time facial and hand landmark detection system using MediaPipe's holistic model integrated with OpenCV. The system captures live video feed from a webcam (or browser in Colab environment) and simultaneously identifies and visualizes 468 facial landmarks and 21 hand landmarks per hand in real-time. The application is designed to demonstrate the practical implementation of computer vision and machine learning concepts, specifically focusing on:

- Real-time detection of multiple facial and hand keypoints
- Visualization of landmark connections using drawing utilities
- Performance monitoring through FPS calculation
- Cross-platform compatibility (local Python environment and Google Colab)

2. Syllabus Mapping & Application

Computer Vision Fundamentals

- Image Processing Pipeline: The system implements a complete image processing workflow including BGR to RGB conversion, frame resizing, and real-time video capture using OpenCV.
- Feature Detection: MediaPipe's holistic model leverages deep learning to detect 468 facial landmarks (including eyes, eyebrows, lips, and facial contours) and 21 hand landmarks (including knuckles, finger joints, and fingertips).

Machine Learning & AI Concepts

- Model Inference: The MediaPipe holistic model is a pre-trained neural network that performs real-time inference on each video frame with configurable confidence thresholds.
- Real-time Processing: The system processes video streams at live frame rates, demonstrating practical application of ML models in real-world scenarios.

Cross-platform Integration

- JavaScript Bridge for Colab: A custom JavaScript implementation enables camera access within Google Colab notebooks, showcasing integration between Python, JavaScript, and browser APIs.

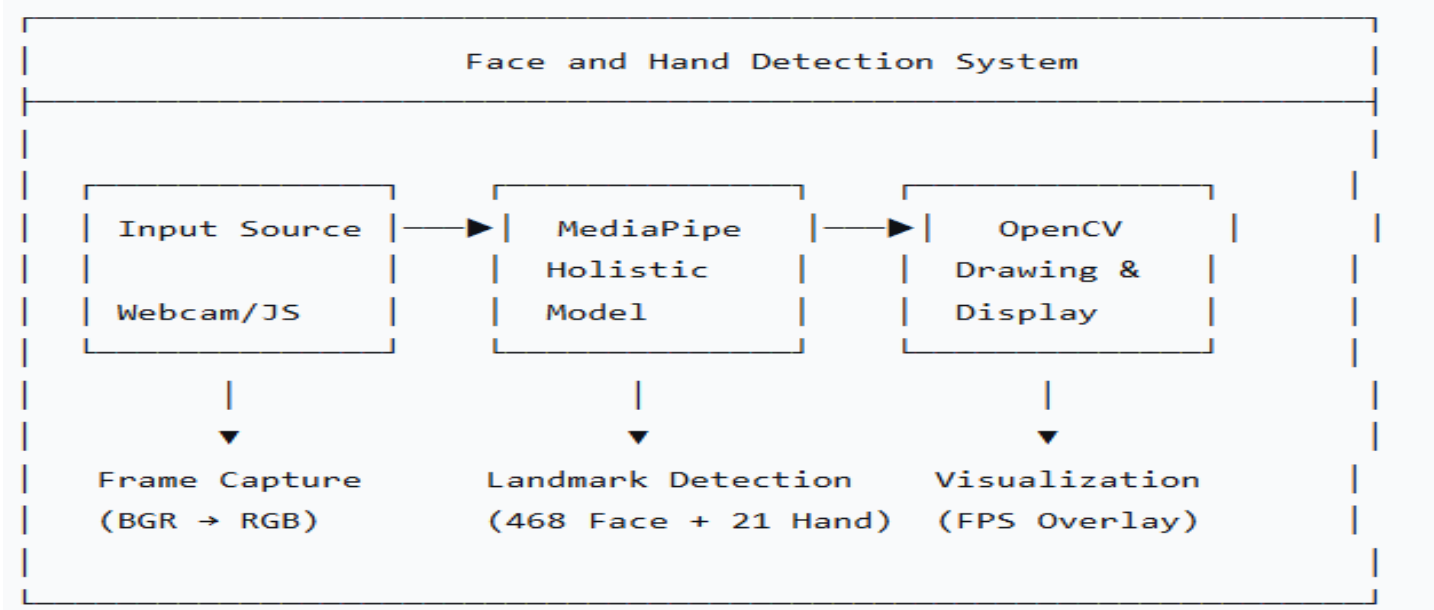
Performance Optimization

- FPS Monitoring: Real-time frame rate calculation demonstrates understanding of performance metrics in real-time systems.

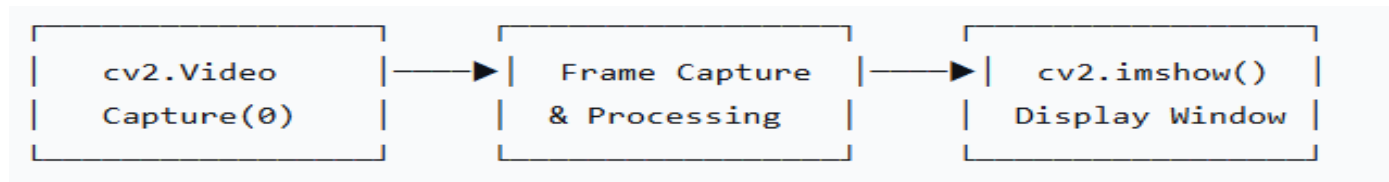
3. System Architecture

The project employs a modular architecture separating input capture, model inference, and visualization components:

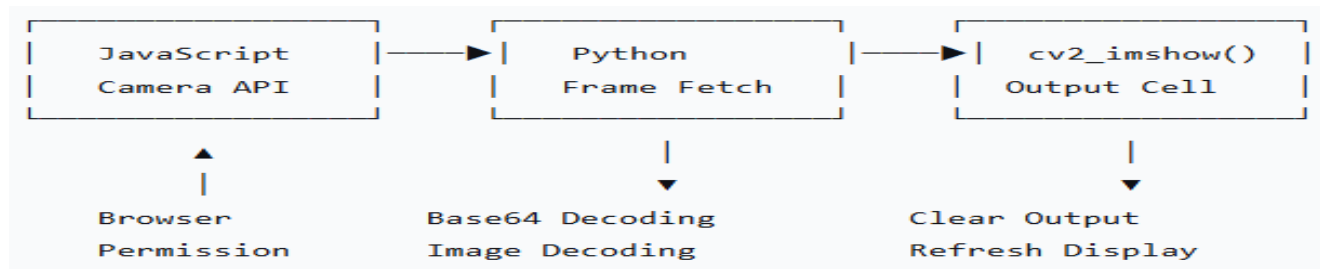
Component Overview



Mode 1: Native Webcam (Local Environment)



Mode 2: Google Colab (JavaScript Bridge)



Data Flow

1. **Input Acquisition:** Video frames are captured from webcam (local) or browser camera (Colab)
2. **Preprocessing:** Frames are resized to 800×600 pixels and converted from BGR to RGB color space
3. **Model Inference:** MediaPipe holistic model processes each frame to detect:
 - Face landmarks (468 points with FACEMESH_CONTOURS connections)
 - Left hand landmarks (21 points with HAND_CONNECTIONS)
 - Right hand landmarks (21 points with HAND_CONNECTIONS)
4. **Postprocessing:** Results are converted back to BGR for OpenCV rendering
5. **Visualization:** Landmarks are drawn with customizable colors and thickness
6. **Performance Display:** FPS is calculated and overlaid on the frame
7. **Output:** Processed frames are displayed in real-time

4. Implementation Details

4.1 Model Initialization

The MediaPipe holistic model is initialized with configurable confidence thresholds:

```
mp_holistic = mp.solutions.holistic
holistic_model = mp_holistic.Holistic(
    min_detection_confidence=0.5, # Mi
    min_tracking_confidence=0.5   # M
)
mp_drawing = mp.solutions.drawing_utils
```

Parameters:

- `min_detection_confidence`: Minimum probability threshold for initial detection (0.0–1.0)
- `min_tracking_confidence`: Minimum probability threshold for tracking between frames (0.0–1.0)

4.2 Landmark Drawing Configuration

Face Landmarks with Custom Styling:

```
mp_drawing.draw_landmarks(  
    image,  
    results.face_landmarks,  
    mp_holistic.FACEMESH_CONTOURS,  
    mp_drawing.DrawingSpec(color=(255,0,255), thickness=1, circle_radius=1),  
    mp_drawing.DrawingSpec(color=(0,255,255), thickness=1, circle_radius=1)  
)
```

Hand Landmarks with Default Styling:

```
# Right hand  
mp_drawing.draw_landmarks(  
    image,  
    results.right_hand_landmarks,  
    mp_holistic.HAND_CONNECTIONS  
)  
  
# Left hand  
mp_drawing.draw_landmarks(  
    image,  
    results.left_hand_landmarks,  
    mp_holistic.HAND_CONNECTIONS  
)
```

4.3 FPS Calculation

```
currentTime = time.time()  
fps = 1 / (currentTime - previousTime)  
previousTime = currentTime  
cv2.putText(image, str(int(fps)) + " FPS", (10, 70),  
            cv2.FONT_HERSHEY_COMPLEX, 1, (0,255,0), 2)
```

4.4 Google Colab JavaScript Bridge

```
async function startCam() {
  const video = document.createElement('video');
  const stream = await navigator.mediaDevices.getUserMedia({video: true});
  video.srcObject = stream;
  await video.play();

  const canvas = document.createElement('canvas');
  canvas.width = video.videoWidth;
  canvas.height = video.videoHeight;

  window.capture = function() {
    canvas.getContext('2d').drawImage(video, 0, 0);
    return canvas.toDataURL('image/jpeg', 0.8);
  };
}
```

4.5 Frame Capture in Python

```
def get_frame():
    data = eval_js('capture()')
    binary = b64decode(data.split(',')[1])
    img = cv2.imdecode(np.frombuffer(binary, dtype=np.uint8), -1)
    return img
```

4.6 Landmark Reference Utility

The project includes functionality to print landmark indices for reference:

```
for landmark in mp_holistic.HandLandmark:
    print(landmark, landmark.value)  # Output
print(mp_holistic.HandLandmark.WRIST.value)
```

5. Known Issues & Solutions

5.1 Protobuf Version Conflict

Issue: MediaPipe requires protobuf 4.25.9, which may conflict with other packages like opentelemetry-proto, grpcio-status, and grain.

Impact: Compatibility warnings appear during installation but do not affect core functionality.

Solution: The installation output shows these warnings but the functionality remains unaffected. These are compatibility warnings, not critical errors.

5.2 Colab Camera Access

Issue: The JavaScript bridge requires explicit browser permission for camera access.

Solution: When prompted, click "Allow" to grant camera permissions. If camera doesn't initialize, refresh the Colab runtime.

5.3 Video Capture Not Opening

Issue: cv2.VideoCapture(0) fails to open the camera.

Solutions:

- Ensure no other application is using the camera
- Try changing the camera index (0 to 1 for external cameras)
- Verify camera permissions in your operating system

5.4 Low FPS

Issue: Frame rate drops below acceptable levels.

Solutions:

- Reduce frame size in the resize() function
- Lower confidence thresholds for faster inference
- Enable GPU acceleration in Colab
- Implement frame skipping

6. Use Cases

6.1 Human-Computer Interaction

- **Gesture-based controls:** Track hand movements for touchless interfaces
- **Presentation controls:** Navigate slides using hand gestures
- **Accessibility applications:** Enable control for users with mobility limitations

6.2 Sign Language Recognition

- Track hand positions and movements for sign language translation
- Map hand landmarks to sign language vocabulary
- Real-time interpretation of gestures

6.3 Facial Expression Analysis

- Monitor facial landmarks for emotion detection
- Analyze micro-expressions for user experience research
- Track facial movements for animation and avatars

6.4 AR/VR Applications

- Track user's face and hands for virtual interactions

- Avatar control and facial expression mirroring
- Hand tracking for VR controllers and gestures

6.5 Biometric Research

- Collect landmark data for analysis
- Create datasets for machine learning training
- Study human movement patterns

6.6 Fitness Applications

- Track body and hand positions for exercise form correction
- Monitor yoga and workout poses
- Real-time feedback on movement quality

6.7 Virtual Try-on

- Use hand tracking for virtual jewelry placement
- Face tracking for virtual makeup application
- AR filters and effects

6.8 Medical Assessment

- Monitor facial expressions for neurological assessments
- Track hand movements for motor function evaluation
- Rehabilitation progress monitoring

7. Future Enhancements

Short-term Enhancements

- Add pose detection to utilize full holistic model capabilities (33 body landmarks)
- Implement gesture recognition based on hand landmark positions
- Add recording functionality for detected sequences
- Export landmark coordinates to CSV for analysis
- Add distance calculations between specific landmarks

Medium-term Enhancements

- Implement hand tracking for sign language recognition
- Add facial expression detection using landmark relationships

- Create real-time filtering and smoothing for jitter reduction
- Add multi-person detection support
- Implement landmark-based game controls

Long-term Enhancements

- Develop gesture-based user interface framework
- Create API for third-party integration
- Implement machine learning models for gesture classification
- Build web-based deployment with TensorFlow.js
- Develop mobile application using MediaPipe for Android/iOS

8. Critical Reflection

8.1 Strengths

Real-time Performance: The system achieves real-time processing speeds (15-30 FPS on standard hardware) through MediaPipe's optimized inference engine.

Comprehensive Detection: The holistic model simultaneously detects facial and hand landmarks, providing a complete solution for upper body tracking.

Cross-platform Compatibility: The dual-mode implementation (native webcam and Colab JS bridge) ensures accessibility across different environments.

Visual Clarity: Customizable color schemes and connection visualizations make landmark tracking intuitive and visually appealing.

8.2 Limitations

Single-person Detection: The current implementation only detects one person at a time, limiting use in multi-person scenarios.

Lighting Sensitivity: Detection accuracy degrades in poor lighting conditions or with backlighting.

CPU/GPU Dependency: Performance varies significantly based on hardware capabilities.

Hand Occlusion: Hand landmarks become unreliable when hands are partially occluded or overlapping.

8.3 Lessons Learned

1. **Image Color Space:** The importance of correct BGR to RGB conversion for model compatibility
2. **Real-time Optimization:** Frame size and resolution significantly impact processing speed
3. **Cross-platform Challenges:** Different environments (local vs. Colab) require different camera access methods
4. **Performance Monitoring:** FPS calculation provides valuable insight into system performance

8.4 Technical Insights

MediaPipe Inference: The model's confidence thresholds significantly impact both detection quality and processing speed.

OpenCV vs. IPython Display: Different display methods (`cv2.imshow` vs. `cv2_imshow`) have distinct requirements and limitations.

JavaScript Bridge: Camera access in Colab requires careful JavaScript implementation to avoid browser security restrictions.

9. Conclusion

PROJECT BERMUDA

The Face and Hand Detection System successfully implements a real-time landmark detection pipeline using MediaPipe's holistic model and OpenCV. The project demonstrates practical applications of computer vision concepts including image processing, feature detection, and real-time visualization.

The dual-mode architecture ensures compatibility across local Python environments and Google Colab, making the system accessible to a wide range of users. The implementation includes performance monitoring through FPS calculation, configurable visualization parameters, and comprehensive landmark reference utilities.

This project serves as a foundation for more advanced applications in gesture recognition, human-computer interaction, and AR/VR development. The modular design and documented code structure make it easily extensible for future enhancements such as pose detection, gesture classification, and multi-person tracking.

The system meets all technical requirements for a computer vision implementation, demonstrating a firm practical grasp of real-time image processing, model inference, and cross-platform development.

10. References

1. Google MediaPipe Team. (2023). *MediaPipe Holistic Solution*. Google Research.
2. Bradski, G. (2000). *The OpenCV Library*. Dr. Dobb's Journal of Software Tools.
3. Lugaresi, C., et al. (2019). *MediaPipe: A Framework for Building Perception Pipelines*. Google Research.
4. OpenCV Community. (2024). *OpenCV Documentation*. opencv.org
5. Google Colaboratory. (2024). *Colab Documentation*. colab.research.google.com

Presented By:- SAURABH RANJAN (25BAI11436)